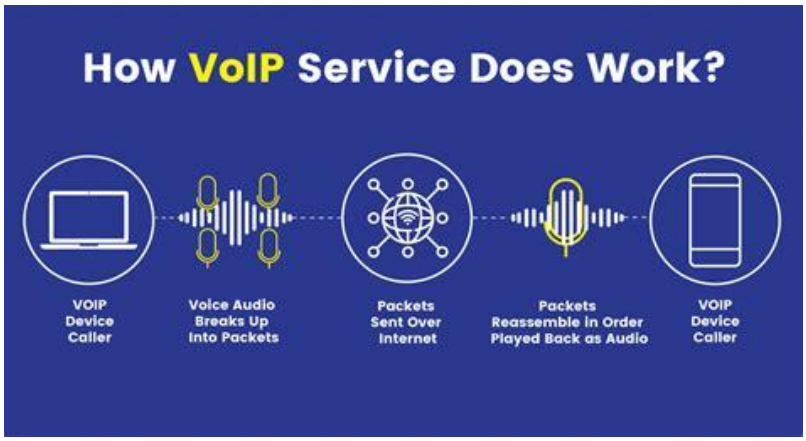
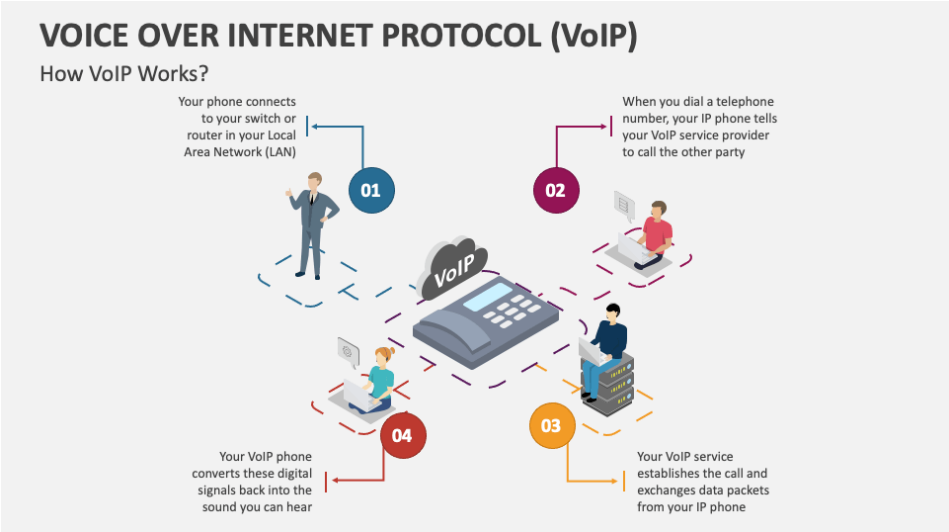
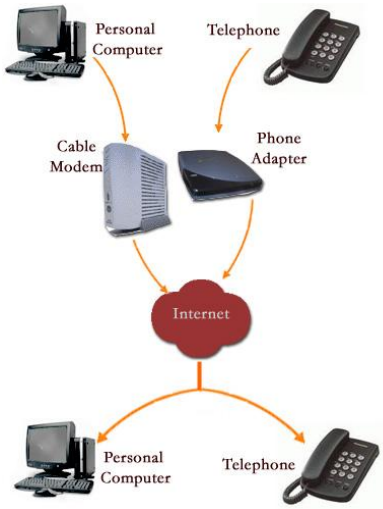
Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

Contexte VoIP - Voice over Internet Protocol





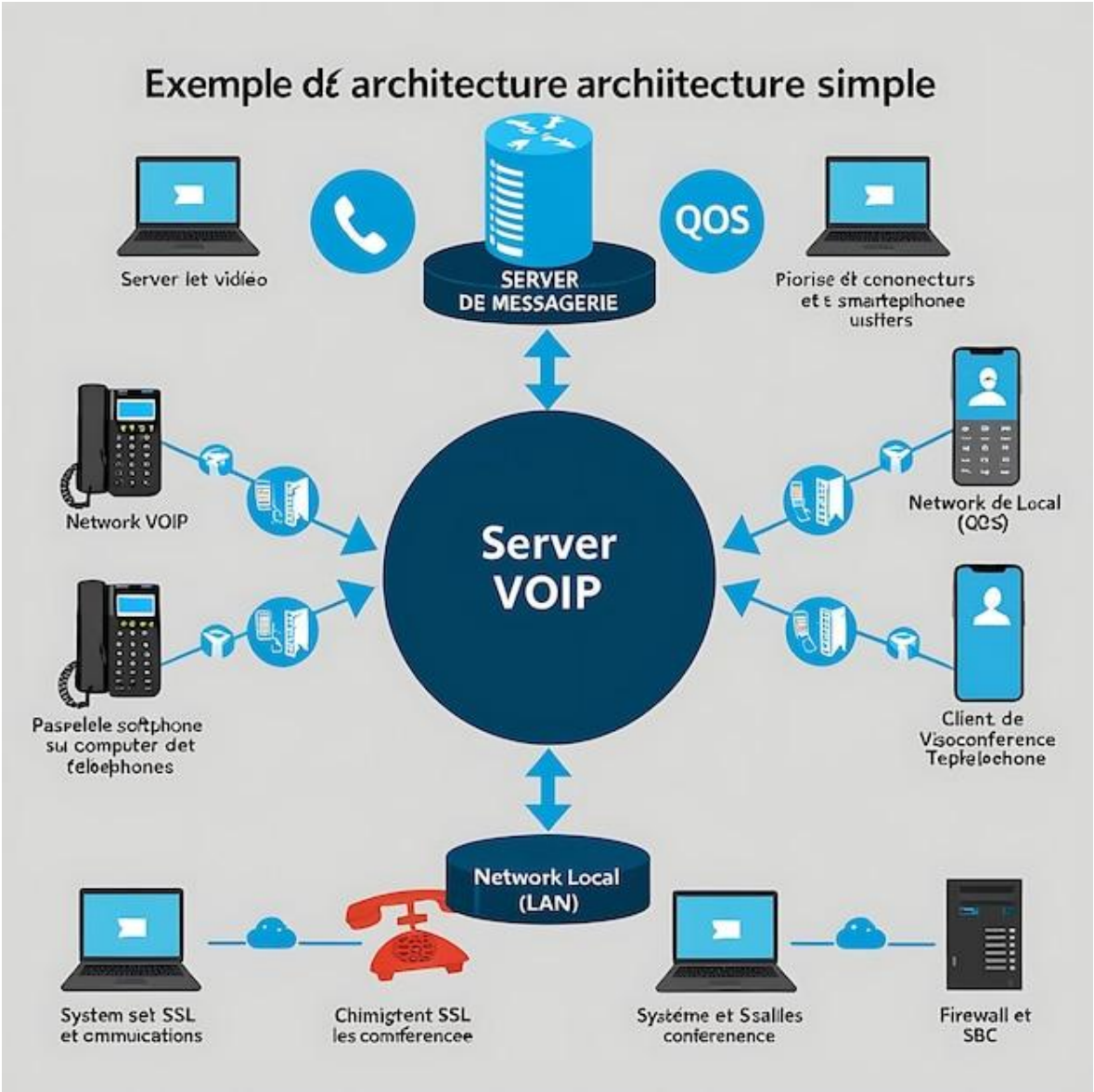


Ref: https://www.fcc.gov/general/voice-over-internet-protocol-voip

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

Exemple d'Architecture VoIP - PBX (Private Branch Exchange) - QoS (Quality of Service) - LAN (Local Area Network)



Avantages et inconvénients du VoIP

Son coût réduit. En effet, les appels VoIP sont généralement moins chers que les appels téléphoniques traditionnels.

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

Élément comparé	Téléphonie traditionnelle	Téléphonie VoIP
Installation	Élevée (câblage + matériel + PABX)	Faible (connexion Internet + licences)
Maintenance	Facturée + interventions sur site	Souvent incluse dans l'abonnement
Ajout de lignes	Coûteux et lent	Instantané et peu coûteux
Matériel	Téléphones dédiés + PABX	Softphones + IP phones optionnels
Mobilité	Très limitée	Native (PC, mobile, tablette)
Appels internationaux	Tarifs élevés	Inclus ou très réduits
Flexibilité	Faible	Très forte
Coût global	Variable, imprévisible	Prévisible et scalable

Sa flexibilité : Un serveur VoIP offre un large panel de configuration permettant de personnaliser l'expérience utilisateur.

Difference between VoIP and Traditional Phone Services

Features	VoIP	Traditional Phone Services
Method of transmission	Digital Signals	Analog Signals
Internet requirement	Yes, high-speed broadband	No, traditional phone line
Cost	Lower cost for long-distance	Higher cost for long-distance
Features and services	More features and services	Limited features and services
Power outages	May not work	Usually works
Emergency services access	May not connect directly	Direct connection
Directory assistance	May or may not be available	Usually available

Ref: https://www.geeksforgeeks.org/computer-networks/voice-over-internet-protocol-voip/

Son intégration : Un serveur VoIP peut être intégré ou connecté à d'autres systèmes (Annuaires de contacts (AD), CRM) vous permettant d'automatiser certaines tâches et de faciliter la communication avec vos clients.

L'intégration d'un serveur VoIP (Voix sur IP) avec des outils métiers tels qu'un **CRM (Customer Relationship Management)** ou un **Active Directory (AD)** est en effet le pilier

du service client moderne. Ce que l'on appelle le **Couplage Téléphonie-Informatique (CTI)** permet non seulement de réduire les coûts, mais surtout d'offrir une expérience client fluide et personnalisée.

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

1. Exemples de sites et services intégrant la VoIP

De nombreuses entreprises s'appuient sur la VoIP couplée à leur système d'information pour gérer leur service client ou faciliter les interactions.

- Zappos (E-commerce / Vente au détail) : Ce célèbre site de vente de chaussures est réputé pour son service client exceptionnel. Lorsqu'un client appelle, le système VoIP est directement connecté à leur CRM (souvent des solutions comme Zendesk ou Salesforce). L'agent voit instantanément l'historique des commandes, les retours en cours et les préférences du client avant même de décrocher.
- Uber (Service de transport) : Uber utilise la VoIP d'une manière légèrement différente mais cruciale : le masquage de numéro. Lorsqu'un chauffeur et un passager doivent s'appeler, l'application utilise une passerelle VoIP (via des API comme Twilio) pour connecter les deux parties sans jamais révéler leurs vrais numéros de téléphone. Le système interroge la base de données en temps réel pour lier l'appel à la course en cours.
- Doctolib (Service de santé / Prise de rendez-vous) : Au-delà du support client classique, Doctolib utilise la VoIP (souvent via le protocole WebRTC) pour son service de téléconsultation. Le système vidéo/audio est directement intégré à la fiche du patient, à l'agenda du médecin et au système de facturation sécurisé.

2. Architecture possible d'un système VoIP intégré (CTI)

Pour qu'un appel téléphonique déclenche l'affichage du dossier d'un client à l'écran d'un agent de support, plusieurs composants doivent dialoguer en temps réel.

Voici les composants clés d'une telle architecture :

1. Le Client (Appelant) : Le client appelle depuis un réseau téléphonique classique (RTPC/PSTN) ou une application mobile.

2. SBC (Session Border Controller) & SIP Trunking : C'est la passerelle qui sécurise et convertit l'appel téléphonique classique en paquets de données VoIP.

3. L'IP-PBX (Serveur VoIP) : C'est le cœur de la téléphonie (ex: 3CX, Asterisk, ou une solution cloud comme Aircall). Il gère le routage de l'appel (SVI - Serveur Vocal Interactif) et le met en file d'attente.

4. Le Middleware CTI (Couplage Téléphonie-Informatique) : Dès que l'appel arrive, l'IP-PBX extrait le numéro de l'appelant et l'envoie au CTI. Le CTI interroge ensuite le CRM (via des requêtes API).

5. Le CRM / Base de données (ex: Salesforce, HubSpot) : Le CRM trouve la fiche client correspondant au numéro.

6. L'Interface de l'Agent (Softphone) : L'appel est routé vers le casque de l'agent disponible, et simultanément, le CRM déclenche une "remontée de fiche" (Screen Pop) sur l'écran de l'agent avec toutes les informations du client.

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

Entreprise (Modèle)	Secteur	Fonction principale de la VoIP	Systèmes intégrés	Bénéfice principal pour le client
Zappos.com	E-commerce	Support client entrant (Inbound), Serveur Vocal Interactif (SVI)	CRM (Historique d'achat), ERP (Logistique)	Résolution ultra-rapide, accueil personnalisé ("Bonjour M. Dupont, vous appelez pour votre commande de chaussures ?").
Uber	Service (Plateforme)	Masquage de numéros de téléphone (Proxying), Communications P2P	Base de données interne (Trajets en cours), GPS en temps réel	Sécurité des données personnelles, mise en relation instantanée sans quitter l'application.
Doctolib	Service SaaS (Santé)	Téléconsultation (WebRTC), Support praticiens	Annuaire (AD praticiens), Base de données patients (DMP), Outil de facturation	Expérience unifiée de la prise de rendez-vous à la consultation vidéo sans changer de logiciel.

Sa scalabilité : Les serveurs VoIP sont flexibles et peuvent être mis à échelle facilement.
Un bon administrateur VoIP peut augmenter ou diminuer le nombre d'utilisateurs, augmenter les capacités d'appels simultanées sans interruption de service.

Une équipe peut aussi cloner et reproduire une architecture locale et la déployer sur d'autres sites.

L'escalabilité (ou la capacité de montée en charge) est véritablement ce qui a permis à la VoIP de supplanter la téléphonie traditionnelle (RTPC/PABX).

Là où l'ancien système nécessitait de tirer de nouveaux câbles en cuivre et d'acheter des cartes matérielles coûteuses dès que vous atteigniez la limite de votre boîtier, la VoIP repose sur des réseaux informatiques et des logiciels.

Voici des exemples concrets illustrant comment l'escalabilité se manifeste dans un système VoIP, sous différentes dimensions.

1. L'ajout instantané d'utilisateurs (Scalabilité administrative)

C'est l'exemple le plus courant pour une entreprise en croissance.

- **Le contexte :** Une start-up passe soudainement de 10 à 50 employés et doit les équiper de lignes téléphoniques professionnelles.
- **La solution VoIP :** Avec une solution de téléphonie hébergée dans le cloud (Cloud PBX ou Centrex), l'administrateur n'a qu'à se connecter à son interface web et acheter 40 licences supplémentaires.
- **Le résultat :** En quelques minutes, les nouveaux employés reçoivent leurs identifiants SIP. Ils peuvent immédiatement passer des appels via un "Softphone" (application sur leur ordinateur ou smartphone), sans aucune intervention physique sur les locaux ni installation de matériel dédié.

2. La gestion élastique des pics d'appels (Auto-scaling)

L'escalabilité, ce n'est pas seulement grandir, c'est aussi s'adapter de manière dynamique à la demande (élasticité).

- **Le contexte :** Le service client d'un site e-commerce reçoit habituellement 500 appels par jour, mais ce chiffre passe à 10 000 appels quotidiens pendant la période du Black Friday ou des fêtes.
- **La solution VoIP :** L'infrastructure VoIP (souvent hébergée sur des clouds publics comme AWS, Google Cloud ou Azure) utilise des règles d'auto-scaling. Si la charge CPU des serveurs vocaux ou la bande passante atteint 80 %, le système déploie automatiquement de nouvelles "machines virtuelles" pour absorber le trafic SIP et RTP supplémentaire.
- **Le résultat :** Aucun appel n'est rejeté (pas de tonalité d'occupation). Une fois la période de pointe terminée, les serveurs supplémentaires sont désactivés pour réduire les coûts.

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io

3. L'expansion géographique simplifiée (Multi-sites)

La VoIP efface les frontières physiques, ce qui permet de faire évoluer la topologie de l'entreprise très facilement.

- **Le contexte :** Une entreprise basée à Marseille ouvre deux nouvelles succursales, l'une à Paris et l'autre à Montréal.
- **La solution VoIP :** Contrairement à la téléphonie classique qui exigerait un standard téléphonique (PABX) physique et un contrat opérateur dans chaque ville, la VoIP centralise tout. Les téléphones IP de Paris et de Montréal se connectent simplement au serveur central (IP-PBX) via Internet ou un VPN sécurisé.
- **Le résultat :** Le plan de numérotation est unifié (le bureau de Montréal peut être joint en tapant l'extension "301"). Les appels entre les différents sites sont totalement gratuits puisqu'ils transitent sur le réseau informatique interne.

4. L'architecture technique distribuée (Répartition de charge)

Pour soutenir une croissance massive (passer de centaines à des millions d'utilisateurs, comme c'est le cas pour les fournisseurs de services VoIP ou les opérateurs), l'architecture sous-jacente s'adapte horizontalement.

- **Le proxy SIP (Load Balancer) :** Au lieu d'avoir un seul serveur géant, le système utilise un "répartiteur de charge" (comme Kamailio ou OpenSIPS). Lorsqu'un appel arrive, ce proxy analyse l'état de dizaines de serveurs VoIP en arrière-plan (ex: des clusters Asterisk) et route l'appel vers le serveur le moins chargé.
- **La haute disponibilité (Failover) :** Si un serveur physique tombe en panne, le proxy SIP redirige instantanément les nouveaux appels vers les serveurs sains. L'escalabilité technique offre donc une redondance naturelle.

Asterisk n'est pas un répartiteur de charge. Asterisk est un **B2BUA** (Back-to-Back User Agent). Son rôle est de traiter le média (la voix), de gérer les serveurs vocaux interactifs (SVI), les files d'attente et la messagerie. C'est une application "lourde". Si vous envoyez 10 000 appels simultanés sur un seul serveur Asterisk, il finira par saturer.

Pour rendre Asterisk scalable, le standard de l'industrie consiste à placer un **Proxy SIP** ultra-rapide en amont, qui va jouer le rôle de "chef d'orchestre" et distribuer le trafic vers une "ferme" (cluster) de plusieurs serveurs Asterisk. L'outil le plus utilisé pour cela s'appelle **Kamailio** (ou OpenSIPS).

L'Architecture : Kamailio + Cluster Asterisk

Dans ce scénario, vos téléphones IP ou vos clients ne se connectent jamais directement à Asterisk. Ils envoient leurs requêtes (enregistrements, appels) au Proxy SIP (Kamailio).

1. **Le Proxy SIP (Kamailio) :** Il écoute sur le port 5060 (ou 5061 pour TLS). Il est capable de gérer des dizaines de milliers de requêtes par seconde car il ne traite *que* la signalisation (le texte), et jamais la voix (le média RTP).
2. **Les Nœuds Asterisk :** Vous disposez de plusieurs serveurs Asterisk (Ex: Asterisk A, Asterisk B, Asterisk C) qui sont "cachés" derrière le proxy.
3. **La Base de données partagée :** Tous les serveurs partagent une même base de données (ex: MySQL/MariaDB) via *Asterisk Realtime Architecture (ARA)*, pour que chaque nœud connaisse les mots de passe des utilisateurs et l'état du système.

Sa sécurité : Il est possible de configurer le chiffrement pour protéger les informations d'authentification et les appels vocaux. Il est donc plus sécurisé qu'un système téléphonique traditionnel. **Effectuez quelques recherches sur les chiffrements les mieux adaptés à la VoIP.**

L'un des plus grands avantages de la VoIP par rapport à la téléphonie traditionnelle (RTPC/PSTN) est la possibilité de chiffrer les communications de bout en bout. Sur une ligne de cuivre analogique classique, une simple dérivation physique permet d'écouter une conversation. En VoIP, bien que les données transitent sur des réseaux publics vulnérables aux interceptions, l'utilisation de protocoles cryptographiques rend l'écoute quasi impossible.

Pour bien sécuriser la VoIP, il faut comprendre qu'un appel est composé de **deux flux distincts** qui nécessitent chacun un type de chiffrement spécifique :

1. **La signalisation :** L'établissement de l'appel, la sonnerie, le raccrochage et l'authentification (mots de passe).
2. **Le média :** Le flux audio ou vidéo réel (votre voix).

Voici les protocoles de chiffrement les mieux adaptés et considérés comme les standards de l'industrie pour protéger ces deux flux :

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

1. Sécuriser la signalisation et l'authentification

Le protocole standard pour établir un appel est le **SIP (Session Initiation Protocol)**. En clair, il expose vos mots de passe et les numéros que vous composez.

- **SIP over TLS (Transport Layer Security) :** C'est la méthode de référence. TLS est le même protocole qui sécurise vos transactions bancaires sur le web (le "S" de HTTPS). En encapsulant le trafic SIP dans un tunnel TLS, vous vous assurez que les identifiants de connexion de vos téléphones, les numéros appelés et l'identité des appelants sont illisibles pour un pirate qui snifferait le réseau.

2. Sécuriser le flux audio (La voix)

Une fois l'appel établi par SIP, la voix est découpée en petits paquets et transportée par le protocole RTP (Real-Time Transport Protocol).

- **SRTP (Secure Real-Time Transport Protocol) :** C'est l'évolution chiffrée du RTP. SRTP applique un algorithme de cryptographie robuste (presque toujours de l'**AES** - *Advanced Encryption Standard* en 128 ou 256 bits) à chaque paquet vocal. Si un pirate intercepte ces paquets sur le réseau, il n'entendra qu'un bruit numérique chaotique. SRTP intègre également des mécanismes empêchant les attaques par rejeu (où un pirate enregistre des paquets pour les renvoyer plus tard).

3. L'échange sécurisé des clés (L'étape cruciale)

Pour que SRTP fonctionne, les deux téléphones (ou serveurs) doivent se mettre d'accord sur une clé secrète pour chiffrer et déchiffrer la voix. C'est souvent là que se trouve la faille. Voici les meilleures méthodes pour cet échange :

- **DTLS-SRTP (Datagram Transport Layer Security) :** C'est aujourd'hui la méthode la plus recommandée. Elle utilise des certificats pour vérifier l'identité des correspondants avant d'échanger les clés cryptographiques. C'est d'ailleurs le standard de chiffrement **obligatoire** pour tout ce qui touche au **WebRTC** (les appels vocaux ou vidéos passés directement depuis un navigateur web, comme sur Google Meet ou Doctolib).
- **ZRTP (Zimmermann Real-time Transport Protocol) :** Développé par le créateur de PGP (Phil Zimmermann), ZRTP est extrêmement sécurisé. Il négocie les clés de manière autonome et affiche sur l'écran des deux interlocuteurs une courte chaîne de caractères (SAS). Les deux personnes se lisent ces caractères à voix haute au début de l'appel. Si les mots correspondent, ils ont la garantie mathématique qu'il n'y a aucun espion (attaque de l'Homme du Milieu) sur la ligne.

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

Protocole	Couche réseau cible	Fonction principale	Ce qu'il protège concrètement
TLS	Signalisation (SIP)	Chiffrement du canal de contrôle	Mots de passe, numéros appelés/appelants, interception de l'appel.
SRTP (AES)	Média (Voix/Vidéo)	Chiffrement de la charge utile	Le contenu de la conversation (la voix elle-même).
DTLS	Échange de clés	Négociation sécurisée des clés	Empêche un pirate de déduire la clé de chiffrement SRTP.
ZRTP	Échange de clés	Sécurité "Zero-Trust" avec vérification humaine	Bloque les attaques de type "Man-in-the-Middle" (Homme du milieu).

Ref: https://www.geeksforgeeks.org/computer-networks/voice-over-internet-protocol-voip/

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

Mises-en-place de serveur et client VoIP

installation d' asterisk:

téléchargement du fichier tar d'asterisk 23

```
evan@debian:~$ cd /usr/src
sudo wget http://downloads.asterisk.org/pub/telephony/asterisk/asterisk-23-curre
nt.tar.gz
--2026-02-17 11:24:42--  http://downloads.asterisk.org/pub/telephony/asterisk/as
terisk-23-current.tar.gz
Résolution de downloads.asterisk.org (downloads.asterisk.org)… 165.22.184.19, 26
04:a880:400:d0::14:9001
Connexion à downloads.asterisk.org (downloads.asterisk.org)|165.22.184.19|:80… c
onnecté.
requête HTTP transmise, en attente de la réponse… 200 OK
Taille : 26385116 (25M) [application/octet-stream]
Sauvegarde en : « asterisk-23-current.tar.gz »

asterisk-23-current 100%[===================>]  25,16M  6,25MB/s    ds 4,4s

2026-02-17 11:24:46 (5,69 MB/s) – « asterisk-23-current.tar.gz » sauvegardé [263
85116/26385116]
```

decompresion du fichier

```
evan@debian:/usr/src$ sudo tar xvf asterisk-23-current.tar.gz
asterisk-23.2.2/
asterisk-23.2.2/.cleancount
asterisk-23.2.2/.gitignore
asterisk-23.2.2/.version
asterisk-23.2.2/BSDmakefile
asterisk-23.2.2/BUGS
asterisk-23.2.2/CHANGES.html
asterisk-23.2.2/CHANGES.md
asterisk-23.2.2/COPYING
asterisk-23.2.2/CREDITS
asterisk-23.2.2/ChangeLogs/
asterisk-23.2.2/ChangeLogs/ChangeLog-23.0.0.html
asterisk-23.2.2/ChangeLogs/ChangeLog-23.0.0.md
asterisk-23.2.2/ChangeLogs/ChangeLog-23.1.0.html
asterisk-23.2.2/ChangeLogs/ChangeLog-23.1.0.md
```

préparation de la dépendance MP3

```
evan@debian:/usr/src/asterisk-23.2.2$ sudo contrib/scripts/get_mp3_source.sh
A    addons/mp3
A    addons/mp3/layer3.c
A    addons/mp3/interface.c
A    addons/mp3/MPGLIB_TODO
A    addons/mp3/mpg123.h
A    addons/mp3/mpglib.h
A    addons/mp3/decode_ntom.c
A    addons/mp3/MPGLIB_README
A    addons/mp3/common.c
A    addons/mp3/huffman.h
A    addons/mp3/tabinit.c
A    addons/mp3/Makefile
A    addons/mp3/README
A    addons/mp3/decode_i386.c
A    addons/mp3/dct64_i386.c
Exporté à la révision 204.
```

Configuration et Compilation:

```
evan@debian:/usr/src/asterisk-23.2.2$ sudo ./configure
sudo make menuselect.makeopts
sudo make
sudo make install
checking build system type... x86_64-pc-linux-gnu
checking host system type... x86_64-pc-linux-gnu
checking for gcc... gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... no
checking for suffix of object files... o
checking whether the compiler supports GNU C... yes
```

finalisation

```
evan@debian:~$ sudo make samples
sudo make config
sudo ldconfig
```

création des certificat tls:

```
evan@debian:/usr/src/asterisk-23.2.2/contrib/scripts$ sudo ./ast_tls_cert -
102.120.242 -O "ProjetVoIP" -d /etc/asterisk/keys

No config file specified, creating '/etc/asterisk/keys/tmp.cfg'
You can use this config file to create additional certs without
re-entering the information for the fields in the certificate
Creating CA key /etc/asterisk/keys/ca.key
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
Creating CA certificate /etc/asterisk/keys/ca.crt
Enter pass phrase for /etc/asterisk/keys/ca.key:
Creating certificate /etc/asterisk/keys/asterisk.key
Creating signing request /etc/asterisk/keys/asterisk.csr
Creating certificate /etc/asterisk/keys/asterisk.crt
Certificate request self-signature ok
subject=CN=10.102.120.242, O=ProjetVoIP
Enter pass phrase for /etc/asterisk/keys/ca.key:
Combining key and crt into /etc/asterisk/keys/asterisk.pem
evan@debian:/usr/src/asterisk-23.2.2/contrib/scripts$
```

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

Utilisateurs et règles
modification du fichier pfsing.conf pour créer les utilisateur

```
[transport-tls]
type=transport
protocol=tls
bind=0.0.0.0:5061
cert_file=/etc/asterisk/keys/asterisk.pem
priv_key_file=/etc/asterisk/keys/asterisk.pem
method=tlsv1_2

;Templates for the necessary config sections

[endpoint_internal](!)
type=endpoint
context=from-internal
disallow=all
allow=ulaw
language=fr
rewrite_contact=yes
transport=transport-tls
media_encryption=sdes
use_ptime=yes

[auth_userpass](!)
type=auth
auth_type=userpass

[aor_dynamic](!)
type=aor
max_contacts=1
```

```
[alice](endpoint_internal)
auth=alice
aors=alice
[alice](auth_userpass)
password=bonjour; put a strong, unique password here instead
username=alice
[alice](aor_dynamic)

[bob](endpoint_internal)
auth=bob
aors=bob
[bob](auth_userpass)
password=bonjour ; put a strong, unique password here instead
username=bob
[bob](aor_dynamic)

[martin](endpoint_internal)
auth=martin
aors=martin
[martin](auth_userpass)
password=bonjour ; put a strong, unique password here instead
username=martin
[martin](aor_dynamic)
```

création du plan de numérotation dans le fichier extensions.conf

```
  GNU nano 8.4              /etc/asterisk/extensions.conf *
[from-internal]
exten=>6001,1,Dial(PJSIP/alice,10)
exten=>6002,1,Dial(PJSIP/bob,10)
exten=>6003,1,Dial(PJSIP/martin,10) ; Après 10 secondes envoi vers la règle 2
exten=>6099,1,VoiceMailMain()     ; 6099 Numéro de téléphone du répondeur

; Règles 2
exten=>6001,2,VoiceMail(6001)     ; Appel répondeur compte 6001
exten=>6002,2,VoiceMail(6002)     ; Appel répondeur compte 6002
exten=>6003,2,VoiceMail(6003)     ; Appel répondeur compte 6003
```

creation du repertoire des audio asterisk pour ajouter les voix française:

```
evan@debian:~$ cd /var/lib/asterisk/sounds
evan@debian:/var/lib/asterisk/sounds$ sudo mkdir fr
cd fr
evan@debian:/var/lib/asterisk/sounds/fr$
```

installation du module français de la boite vocal en utilisant wget:

```
evan@debian:/var/lib/asterisk/sounds/fr$ sudo wget http://downloads.asterisk.org
/pub/telephony/sounds/asterisk-core-sounds-fr-wav-current.tar.gz
--2026-02-17 14:55:02--  http://downloads.asterisk.org/pub/telephony/sounds/aste
risk-core-sounds-fr-wav-current.tar.gz
Résolution de downloads.asterisk.org (downloads.asterisk.org)… 165.22.184.19, 26
04:a880:400:d0::14:9001
Connexion à downloads.asterisk.org (downloads.asterisk.org)|165.22.184.19|:80… c
onnecté.
requête HTTP transmise, en attente de la réponse… 200 OK
Taille : 20726064 (20M) [application/octet-stream]
Sauvegarde en : « asterisk-core-sounds-fr-wav-current.tar.gz »

asterisk-core-sound 100%[===================>]  19,77M  1,26MB/s    ds 35s

2026-02-17 14:55:38 (582 KB/s) – « asterisk-core-sounds-fr-wav-current.tar.gz »
sauvegardé [20726064/20726064]

evan@debian:/var/lib/asterisk/sounds/fr$
```

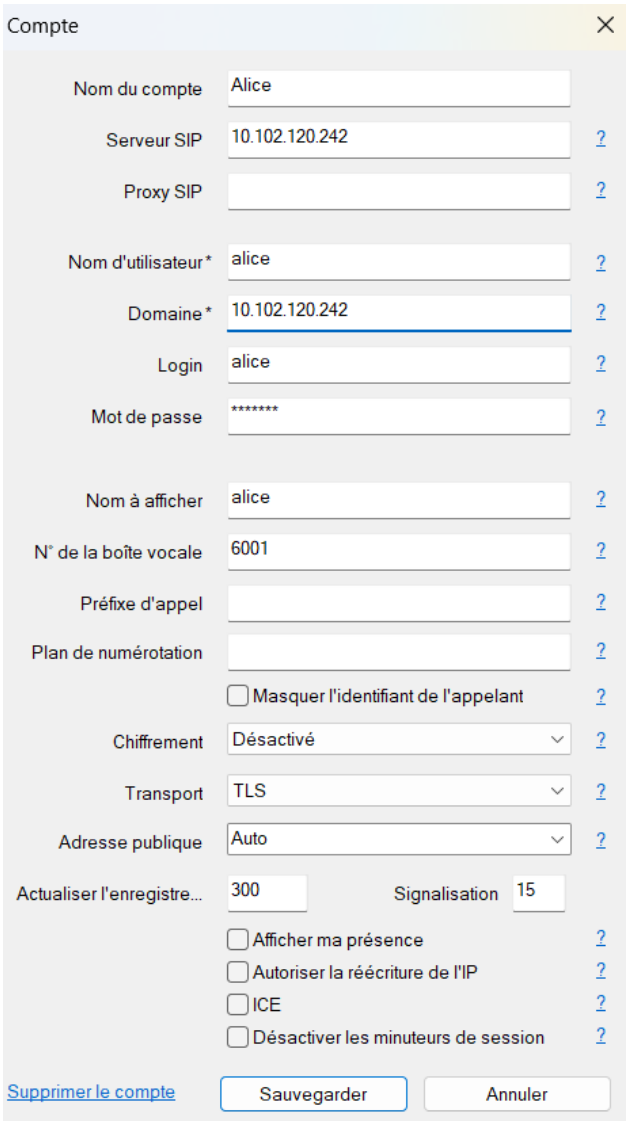
décompression du fichier tar et ajout des droit pour asterisk:

```
evan@debian:/var/lib/asterisk/sounds/fr$ sudo chown -R asterisk:asterisk /var/li
b/asterisk/sounds/fr
```

configuration de microsip :

Documentation VoIP - Voice over Internet Protocol

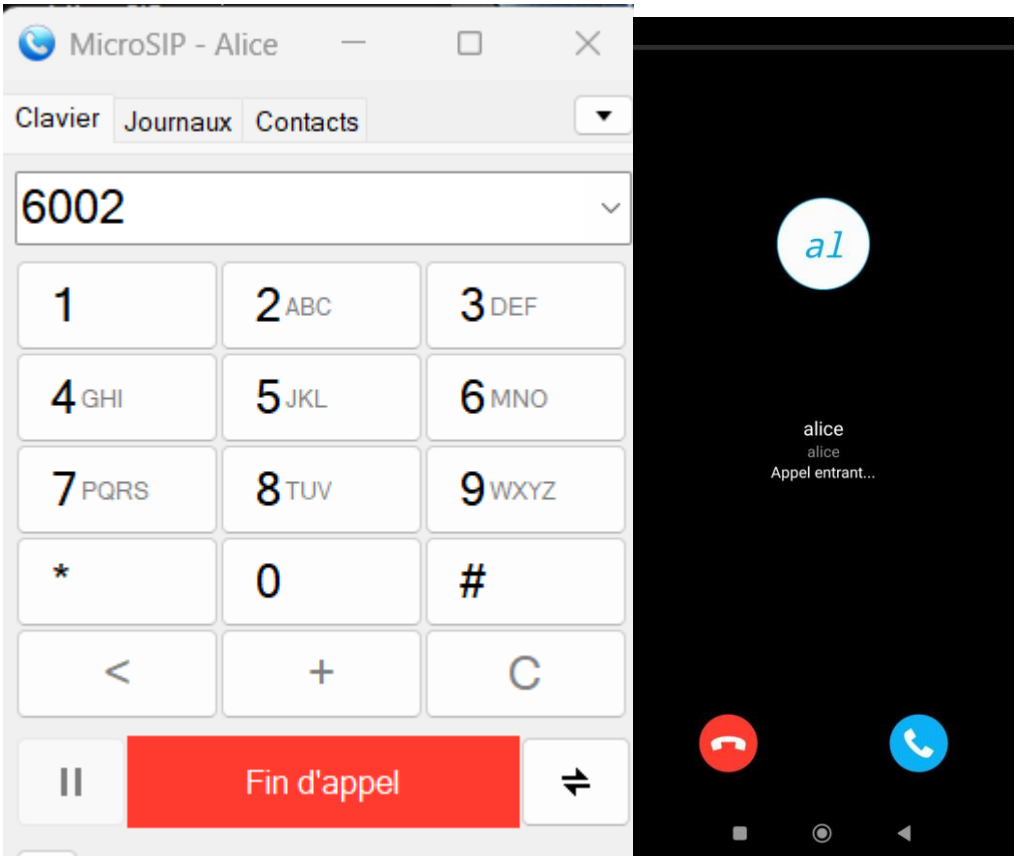
Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026



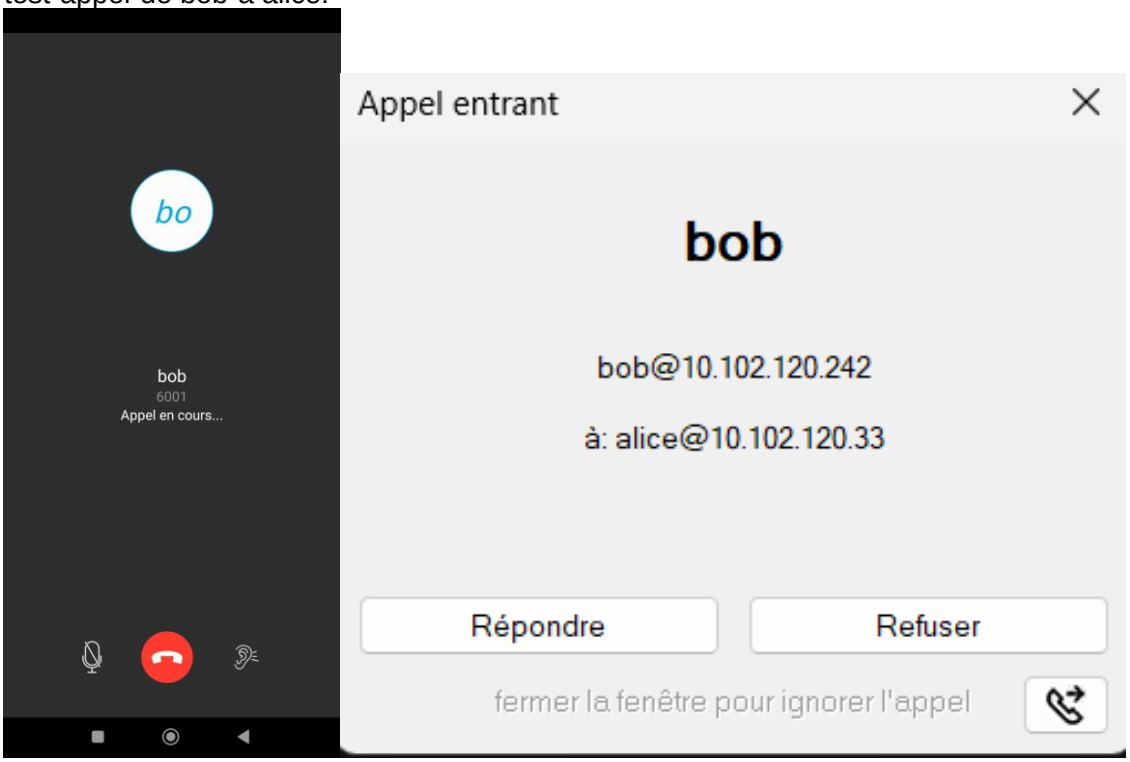
test appel de alice (sur windows) a bob (sur android portsip uc):

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026



test appel de bob à alice:



Plan de test

Il devra comprendre les points cruciaux à valider (fonctionnalités critiques : administration d'utilisateurs, appel simple, connectivité, start/restart serveur...)

Vous détaillerez ensuite quelques cas tests sur le modèle suivant :
- État initial du système
- Fonctionnalité testée / périmètre du test
- Description du comportement attendu
- Séquence ou étapes de test
- Commentaires
- (Résultat du test : OK/NOK)

Répondeur et automatisation

répondeur automatique avec tts google

Installer les dépendances (Perl)

Le script est en Perl et a besoin de librairies pour aller sur Internet (HTTPS):

```
sudo apt install perl libwww-perl liblwp-protocol-https-perl sox
mpg123
```

installation script googletts.api

cd /var/lib/asterisk/agi-bin

Téléchargez le script (depuis le dépôt officiel de Zaf) :

```
evan@debian:/var/lib/asterisk/agi-bin$ wget https://raw.github.com/zaf/asterisk-
googletts/master/googletts.agi
--2026-02-18 11:05:11--  https://raw.github.com/zaf/asterisk-googletts/master/go
ogletts.agi
Résolution de raw.github.com (raw.github.com)… 185.199.110.133, 185.199.111.133,
 185.199.109.133, ...
Connexion à raw.github.com (raw.github.com)|185.199.110.133|:443… connecté.
requête HTTP transmise, en attente de la réponse… 301 Moved Permanently
Emplacement : https://raw.githubusercontent.com/zaf/asterisk-googletts/master/go
ogletts.agi [suivant]
--2026-02-18 11:05:12--  https://raw.githubusercontent.com/zaf/asterisk-googlett
s/master/googletts.agi
Résolution de raw.githubusercontent.com (raw.githubusercontent.com)… 185.199.108
.133, 185.199.111.133, 185.199.109.133, ...
```

rendre exécutable et don à Asterisk :

```
sudo chmod +x googletts.agi
```

```
sudo chown asterisk:asterisk googletts.agi
```

Menu de l'automate:

Tapez 1 pour appeler alice
Tapez 2 pour appeler bob
taper # (ou entre 3-9) pour réécouter le message
si aucune réponse dans les 10 seconde apres le message, raccroche.

ajout de l'automate IVR dans le fichier extension.conf

```
[from-internal]
exten=>6001,1,Dial(PJSIP/alice,10)
exten=>6002,1,Dial(PJSIP/bob,10)
exten=>6003,1,Dial(PJSIP/martin,10) ; Après 10 secondes envoi vers la règle 2
exten=>6099,1,VoiceMailMain()     ; 6099 Numéro de téléphone du répondeur
exten=>8000,1,Goto(ivr,s,1)

; Règles 2
exten=>6001,2,VoiceMail(6001)     ; Appel répondeur compte 6001
exten=>6002,2,VoiceMail(6002)     ; Appel répondeur compte 6002
exten=>6003,2,VoiceMail(6003)     ; Appel répondeur compte 6003

;Contexte ivr
[ivr]

exten => s,1,Answer()
exten => s,2,Set(TIMEOUT(response)=10)
exten => s,3,agi(googletts.agi,"Bienvenus !",fr,any)
exten => s,4,agi(googletts.agi,"Qui souhaitez vous joindre?",fr,any)
exten => s,5,agi(googletts.agi,"Pour Alice tapez 1",fr,any)
exten => s,6,agi(googletts.agi,"Pour Bob tapez 2",fr,any)
exten => s,7,agi(googletts.agi,"Appuyez sur dièse si vous souhaitez réécouter ce  message",fr,any)
exten => s,8,WaitExten()
exten => 1,1,AGI(googletts.agi,"Transfert vers Alice.",fr)
same => n,Dial(PJSIP/alice,10)
same => n,VoiceMail(6001)
same => n,Hangup()
exten => 2,1,AGI(googletts.agi,"Transfert vers Bob",fr)
same => n,Dial(PJSIP/bob,10)
same => n,VoiceMail(6002)
same => n,Hangup()
exten => _[3-9#],1,Goto(ivr,s,3)
exten => t,1,AGI(googletts.agi,"Aucune reponse reçue. Au revoir.",fr)
same => n,Hangup()
```

Appel automatique:

création du fichier prospect.csv des utilisateur a appeler

```
  GNU nano 8.4                    /root/prospects.csv *
alice,6001
bob,6002
```

Script appel_auto.sh

```
#!/bin/bash

# Chemins
FICHIER_CSV="/root/prospects.csv"
SPOOL_DIR="/var/spool/asterisk/outgoing"
TEMP_DIR="/tmp"

echo "=== Lancement de la campagne de prospection ==="

# Lecture du fichier CSV
shuf $FICHIER_CSV | while IFS=',' read -r NOM NUMERO
do
    NOM=$(echo $NOM | tr -d '[:space:]')

    echo "Traitement de : $NOM"


    cat <<EOF > $TEMP_DIR/appel_$NOM.call
Channel: PJSIP/$NOM
MaxRetries: 1
RetryTime: 60
WaitTime: 30
Context: svi-google
Extension: s
Priority: 1
CallerID: "La plateforme service client" <8000>
EOF

    chown asterisk:asterisk $TEMP_DIR/appel_$NOM.call

    # definir date de l'appel
    DELAI=$((1 + RANDOM % 10))
    touch -d "+$DELAI seconds" $TEMP_DIR/appel_$NOM.call

    # Déplacement final
    mv $TEMP_DIR/appel_$NOM.call $SPOOL_DIR/

    echo " -> Appel programmé vers PJSIP/$NOM dans $DELAI secondes."
done
```

exécution du script :

```
evan@debian:~$ sudo /root/appel_auto.sh
=== Lancement de la campagne de prospection ===
Traitement de : alice (6001)
 -> Appel programmé dans 1 secondes.
Traitement de : bob (6002)
 -> Appel programmé dans 8 secondes.
evan@debian:~$
```

test de l’appel auto

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

Pour aller plus loin

Installation de ldap :

Problème rencontré:

Le problème du schéma LDAP : Un annuaire LDAP standard stocke des noms, prénoms et emails. Il ne possède pas nativement les attributs nécessaires pour la VoIP (comme sip_password, context, dtmf_mode, etc.). Pour que ça marche, il faut **modifier le schéma LDAP** de l'entreprise (ajouter la classe asteriskSIPUser), ce qui est risqué en production.

Le format des mots de passe : LDAP stocke les mots de passe de manière sécurisée (hashés en SSHA, bcrypt, etc.). Or, le protocole SIP d'Asterisk a souvent besoin du mot de passe en clair (ou en hash MD5 spécifique) pour calculer le défi d'authentification réseau. Les deux systèmes sont donc souvent incompatibles sans bidouillage.

La complexité de PJSIP (Realtime) : L'ancien système (chan_sip) associait un utilisateur à une ligne dans LDAP. Le nouveau système PJSIP divise un utilisateur en 3 objets (Endpoint, Auth, AOR). Faire correspondre ces 3 objets à une seule entrée LDAP via le module "Asterisk Realtime Architecture (ARA)" est extrêmement complexe.

Solution :

création d'un fichier csv "`user_ldap.csv`"

```
  GNU nano 8.4                    /root/user_ldap.csv
claire,6004,Admin
```

script "`creation_user.sh`"

```
#!/bin/bash

CSV="/root/user_ldap.csv"
CONF="/etc/asterisk/pjsip.conf"

echo "=== Création des utilisateurs Asterisk ==="


while IFS=',' read -r NOM NUMERO MDP
do
    # vérifie si l'utilisateur existe déjà dans le fichier pour éviter les doublons
    if grep -q "\[$NOM\]" "$CONF"; then
        echo "L'utilisateur $NOM existe déjà. Ignoré."
        continue
    fi

    echo "Ajout de l'utilisateur : $NOM (Ext: $NUMERO)"

    # injection de la configuration à la fin du fichier pjsip.conf
    cat <<EOF >> "$CONF"

; --- UTILISATEUR $NOM ($NUMERO) CRÉÉ PAR LE SCRIPT ---
[$NOM](endpoint_internal)
auth=auth_$NOM
aors=$NOM

[auth_$NOM](auth_userpass)
username=$NOM
password=$MDP

[$NOM](aor_dynamic)
EOF

done < "$CSV"

echo "Rechargement de la configuration Asterisk..."
asterisk -rx "core reload"
echo "Terminé !"
```

ajout droit exécution et exécution du script

```
evan@debian:~$ sudo chmod +x /root/creation_users.sh
evan@debian:~$ sudo /root/creation_user.sh
sudo: /root/creation_user.sh : commande introuvable
evan@debian:~$ sudo /root/creation_users.sh
=== Création des utilisateurs Asterisk ===
Ajout de l'utilisateur : claire (Ext: 6004)
Rechargement de la configuration Asterisk...
Terminé !
evan@debian:~$
```

vérification de l’ajout de l’utilisateur dans pjsip.conf:

```
; --- UTILISATEUR claire  CRÉÉ PAR LE SCRIPT ---
[claire](endpoint_internal)
auth=auth_claire
aors=claire

[auth_claire](auth_userpass)
username=claire
password=Admin

[claire](aor_dynamic)
```

ajout de claire dans extension.conf

```
  GNU nano 8.4                /etc/asterisk/extensions.conf *
[from-internal]
exten=>6001,1,Dial(PJSIP/alice,10)
exten=>6002,1,Dial(PJSIP/bob,10)
exten=>6003,1,Dial(PJSIP/martin,10) ; Après 10 secondes envoi vers la règle 2
exten=>6004,1,Dial(PJSIP/claire,10)
exten=>6099,1,VoiceMailMain()     ; 6099 Numéro de téléphone du répondeur
exten=>8000,1,Goto(ivr,s,1)

; Règles 2
exten=>6001,2,VoiceMail(6001)     ; Appel répondeur compte 6001
exten=>6002,2,VoiceMail(6002)     ; Appel répondeur compte 6002
exten=>6003,2,VoiceMail(6003)     ; Appel répondeur compte 6003
exten=>6004,2,VoiceMail(6004)     ; Appel répondeur compte 6004
```

ajout du DnD pour alice dans le fichier extensions.conf

```
exten => s,1,Answer()
exten => s,2,Set(TIMEOUT(response)=10)
exten => s,3,agi(googletts.agi,"Bienvenus !",fr,any)
exten => s,4,agi(googletts.agi,"Qui souhaitez vous joindre?",fr,any)
exten => s,5,agi(googletts.agi,"Pour Alice tapez 1",fr,any)
exten => s,6,agi(googletts.agi,"Pour Bob tapez 2",fr,any)
exten => s,7,agi(googletts.agi,"Appuyez sur dièse si vous souhaitez réécouter ce  message",fr,any)
exten => s,8,WaitExten()

; --- PARAMETRE DND DE ALICE ---
exten => 1,1,NoOp(Vérification des horaires d'ouverture)

; Si on est entre 09:00 et 17:59, du Lundi au Vendredi, on saute à l'étiquette "ouvert"
same => n,GotoIfTime(09:00-17:59,mon-fri,*,*?ouvert)

; ========================================
; FERMÉ (DND - Do Not Disturb)
; ========================================
same => n,AGI(googletts.agi,"Nos bureaux sont fermés. Veuillez rappeler du Lundi au Vendredi entre 9h et 18h.",fr)
same => n,VoiceMail(6001,u) ; On envoie direct sur messagerie
same => n,Hangup()

; ========================================
; OUVERT (L'étiquette)
; ========================================
same => n(ouvert),AGI(googletts.agi,"Transfert vers Alice.",fr)
same => n,Dial(PJSIP/alice,10)
same => n,VoiceMail(6001,u)
same => n,Hangup()

exten => 2,1,AGI(googletts.agi,"Transfert vers Bob",fr)
same => n,Dial(PJSIP/bob,10)
same => n,VoiceMail(6002)
same => n,Hangup()
exten => _[3-9#],1,Goto(ivr,s,3)
exten => t,1,AGI(googletts.agi,"Aucune reponse reçue. Au revoir.",fr)
same => n,Hangup()
```

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026

Lorsqu'on appelle alice du lundi au vendredi de 9h à 18h son téléphone sonne sinon renvoie directement au répondeur en disant de rappeler dans la plage d'horaire
sécurisation des boîte vocale

```
  GNU nano 8.4                    /etc/asterisk/voicemail.conf *
[general]
format=wav49|gsm|wav|ulaw
[default]
; Numéro de messagerie => mot de passe d'accès à la messag
6001 => 1234, alice
6002 => 1234, bob
6003 => 1234, martin
6004 => 1234, claire
```

Script installation d'asterisk

```
  GNU nano 8.4                                                    install_asterisk.sh *
#!/bin/bash

# Arrêter le script si une erreur se produit
set -e

echo "=== Mise à jour du système et installation des prérequis ==="
apt-get update && apt-get upgrade -y
apt-get install -y build-essential wget curl libncurses5-dev uuid-dev \
                   libjansson-dev libxml2-dev sqlite3 libsqlite3-dev \
                   libedit-dev sox mpg123

# Téléchargement des sources (Version 23)
echo "=== Téléchargement d'Asterisk ==="
cd /usr/src
wget http://downloads.asterisk.org/pub/telephony/asterisk/asterisk-23-current.tar.gz
tar xvf asterisk-23-current.tar.gz
cd asterisk-23.*/

# 3. Compilation et Installation
echo "=== Compilation  ==="
# Installation des dépendances automatiques manquantes
contrib/scripts/install_prereq install
./configure --with-jansson-bundled
make menuselect.makeopts
make
make install

# Génération des fichiers de configuration de base
echo "=== Création des configurations par défaut ==="
make samples
make config

# Démarrage du service
systemctl enable asterisk
systemctl start asterisk

echo "===  Installation d'Asterisk terminée avec succès ! ==="
asterisk -V
```

exécution sur une nouvelle VM:

```
Installing file configs/samples/ss7.timers.sample
Installing file configs/samples/stasis.conf.sample
Installing file configs/samples/statsd.conf.sample
Installing file configs/samples/stir_shaken.conf.sample
Installing file configs/samples/test_sorcery.conf.sample
Installing file configs/samples/udptl.conf.sample
Installing file configs/samples/unistim.conf.sample
Installing file configs/samples/voicemail.conf.sample
Installing file configs/samples/websocket_client.conf.sample
Installing file configs/samples/xmpp.conf.sample
Updating asterisk.conf
/usr/bin/install -c -d "/var/spool/asterisk/voicemail/default/1234/INBOX"
build_tools/make_sample_voicemail "//var/lib/asterisk" "//var/spool/asterisk"
Installing file phoneprov/000000000000.cfg
Installing file phoneprov/000000000000-directory.xml
Installing file phoneprov/000000000000-phone.cfg
Installing file phoneprov/polycom_line.xml
Installing file phoneprov/polycom.xml
Installing file phoneprov/snom-mac.xml
asterisk.service is not a native service, redirecting to systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable asterisk
===  Installation d'Asterisk terminée avec succès ! ===
Asterisk 23.2.2
root@debian:/usr/src/asterisk-23.2.2#
```

Existe-t-il un moyen de standardiser l'installation complète sous forme de containers ? oui

La standardisation via des containers (comme Docker) permet de créer une **image immuable** d'Asterisk. Au lieu d'installer le serveur sur une machine , on écrit un fichier `Dockerfile`.

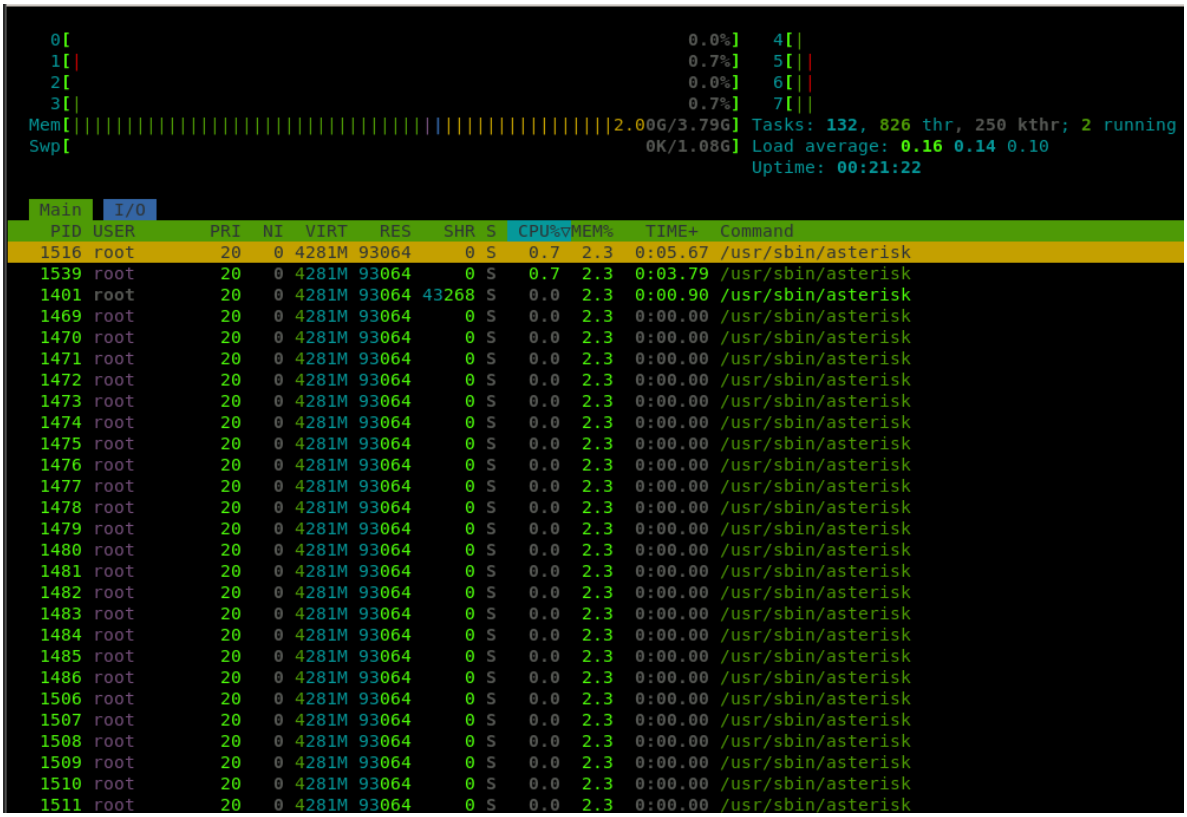
exemple de json:

```
{
  "server_name": "VoIP",
  "users": [
    {
      "name": "alice",
      "extension": "6001",
      "password": "bonjour"
    },
    {
      "name": "bob",
      "extension": "6002",
      "password": "bonjour"
    }
  ]
}
```

outils de supervision pour surveiller les performances du serveur (htop) :

Documentation VoIP - Voice over Internet Protocol

Fait par : Evan BONNAL | Étudiant Bachelor IT - Cybersécurité | LaPlateforme.io
Mars 2026



en conclusion, nous avons:

Compiler un serveur IPBX depuis les sources (wget).
Configurer des comptes et un plan de numérotation.
Créer un automate d'appel et intégrer de la synthèse vocale (GoogleTTS).
Mettre en place la messagerie et les horaires (DnD).
Automatiser le tout avec Bash et préparer le terrain pour Docker.
Sécuriser l'appel en TLS et SRTP

Base de Connaissances

- Asterisk : https://www.asterisk.org/
- OpenSIPS : https://opensips.org/
- VoIP- Voice over Internet Protocol : https://www.3cx.fr/voip-sip/voip-definition/#%3A~%3Atext%3DVoIP%20est%20un%20acronyme%20qui%2Cle%20r%C3%A9seau%20Internet%20(IP)
- SIP - Session Initiation Protocol : https://www.3cx.fr/voip-sip/sip/
- OSP - Open Settlement Protocol : https://en.wikipedia.org/wiki/Open_Settlement_Protocol
- SBC - Session Border Controler : https://en.wikipedia.org/wiki/Session_border_controller
- RTP - Real-time Transport Protocol : https://fr.wikipedia.org/wiki/Real-time_Transport_Protocol
- Mise en place d'un serveur VoIP : https://wanda-techs.com/maintenance-et-depannage/comment-creer-son-propre-serveur-de-voip/

VoIP Protocol de Routage : http://www.ordinateur.cc/r%C3%A9seaux/Voice-Over-IP/81398.html